

CENTRO UNIVERSITÁRIO — ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Controle Financeiro Pessoal

Documentação Técnica do Trabalho de Conclusão de Curso

Integrantes do grupo:

[Preencher: Nome Completo — Matrícula]

[Preencher: Nome Completo — Matrícula]

Backend: Java 21 · Spring Boot · PostgreSQL | Frontend: React · TypeScript · Vite
junho de 2026

Sumário

1. Documento de Requisitos
2. Diagrama de Casos de Uso
3. Modelo Lógico Relacional
4. Diagrama de Classes
5. Diagramas de Sequência
6. Arquitetura e Segurança
7. Segurança — Detalhamento Técnico
8. Arquitetura do Frontend
9. Testes Automatizados
10. Apêndice — Renda Mensal Automática

Documento de Requisitos

Sistema: **Controle Financeiro Pessoal** Tipo: Aplicação web (API REST + SPA)

1. Objetivo

Permitir que uma pessoa registre suas **receitas** e **despesas**, organize-as em **categorias** e acompanhe sua situação financeira por meio de um **painel** com indicadores e gráficos. O sistema também lança automaticamente a **renda mensal** do usuário como receita.

2. Escopo

O sistema cobre o controle financeiro individual: cada usuário acessa apenas os próprios dados. Está **fora do escopo** desta versão: contas compartilhadas, integração bancária (Open Finance), metas, simuladores, importação de extratos e notificações por e-mail.

3. Requisitos Funcionais (RF)

Código	Requisito	Descrição
RF01	Cadastrar usuário	O sistema deve permitir criar uma conta com nome, e-mail, senha e renda mensal (opcional).
RF02	Autenticar usuário	O sistema deve autenticar por e-mail e senha, retornando um token de acesso.
RF03	Impedir e-mail duplicado	O sistema não deve permitir duas contas com o mesmo e-mail.
RF04	Categorias padrão	Ao criar a conta, o sistema deve cadastrar categorias iniciais automaticamente.
RF05	Gerenciar categorias	O usuário deve poder criar, listar, editar e excluir categorias (receita ou despesa).
RF06	Proteger exclusão de categoria	O sistema não deve excluir categoria que possua transações vinculadas.
RF07	Gerenciar transações	O usuário deve poder criar, listar (paginado), editar e excluir receitas e despesas.
RF08	Vincular transação a categoria	Toda transação deve pertencer a uma categoria do próprio usuário.
RF09	Visualizar dashboard	O sistema deve exibir saldo atual, receitas e despesas do mês, gastos por categoria e fluxo de caixa de 6 meses.
RF10	Filtrar dashboard por período	O usuário deve poder consultar o resumo de um mês específico (yyyy-MM).
RF11	Gerenciar perfil	O usuário deve poder visualizar e editar nome e renda mensal.
RF12	Alterar senha	O usuário deve poder trocar a senha, confirmando a senha atual.
RF13	Lançar renda mensal automaticamente	O sistema deve lançar a renda do usuário como receita no 5º dia útil de cada mês.
RF14	Evitar renda duplicada	O sistema deve garantir um único lançamento de renda por usuário/mês.
RF15	Consultar status da renda	O usuário deve poder ver se a renda do mês já foi lançada e a próxima data prevista.

4. Requisitos Não Funcionais (RNF)

Código	Requisito	Descrição
RNF01	Segurança da senha	As senhas devem ser armazenadas com hash BCrypt (salt automático), nunca em texto puro.
RNF02	Autenticação stateless	A autenticação deve usar JWT , sem manter sessão no servidor.

Código	Requisito	Descrição
RNF03	Isolamento de dados	Cada usuário deve acessar exclusivamente os próprios dados.
RNF04	Validação de entrada	As requisições devem ser validadas (campos obrigatórios, formatos) antes do processamento.
RNF05	Documentação da API	A API deve expor documentação interativa (Swagger/OpenAPI).
RNF06	Portabilidade	O sistema deve rodar em qualquer ambiente com Java 21, Node e PostgreSQL.
RNF07	Usabilidade	A interface deve ser responsiva e exibir mensagens de erro claras ao usuário.
RNF08	Disponibilidade em nuvem	O backend deve tolerar <i>cold start</i> em hospedagem gratuita (espera e endpoint de health).
RNF09	Desempenho de listagem	A listagem de transações deve ser paginada para suportar grande volume.

5. Regras de Negócio (RN)

Código	Regra
RN01	O saldo atual é a soma de todas as receitas menos todas as despesas do histórico.
RN02	O lançamento automático de renda ocorre no 5º dia útil (segunda a sexta; feriados não considerados).
RN03	A renda mensal só é lançada se for maior que zero.
RN04	Se a renda mudar no perfil, o lançamento do mês corrente é atualizado.
RN05	O percentual de cada categoria no dashboard é calculado sobre o total de despesas do mês.
RN06	O fluxo de caixa considera os 6 meses até o período selecionado.

6. Atores

Ator	Papel
Visitante	Usuário não autenticado; acessa apenas cadastro e login.
Usuário	Usuário autenticado; gerencia categorias, transações e perfil.
Agendador	Componente de sistema que dispara o lançamento automático da renda.

Diagrama de Casos de Uso

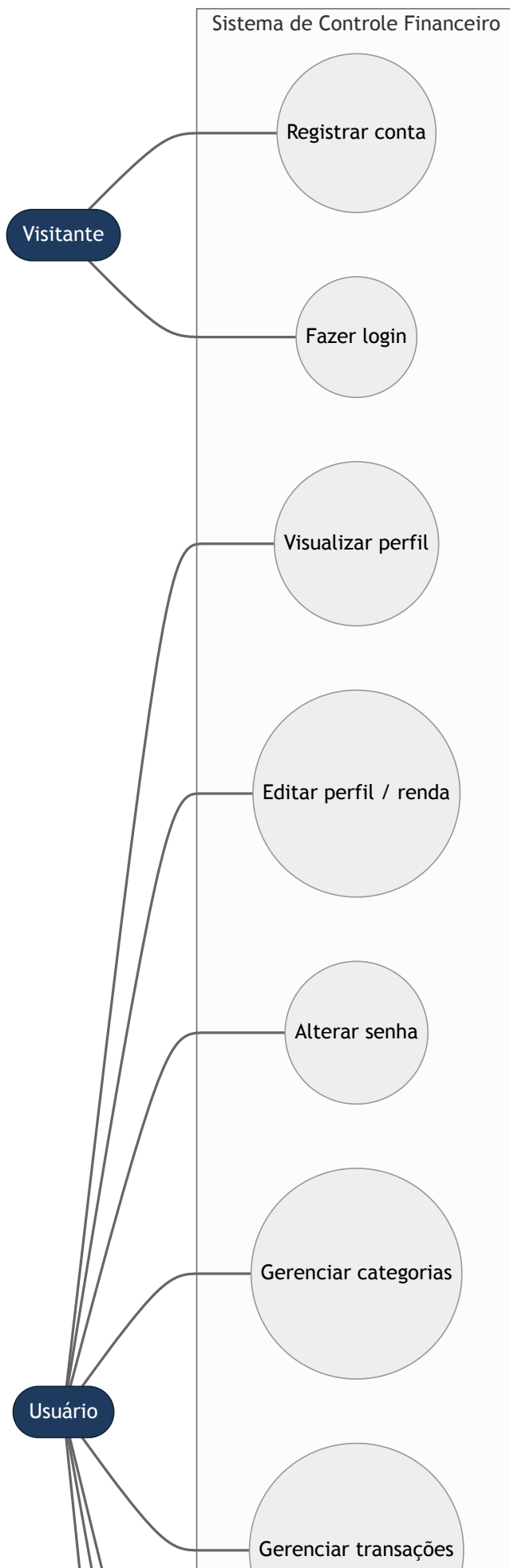
O diagrama de casos de uso descreve o sistema sob a ótica de **quem o utiliza** e **o que cada ator consegue fazer**. Ele não detalha implementação: serve para delimitar o escopo funcional do Controle Financeiro.

Atores

Ator	Descrição
Visitante	Pessoa ainda não autenticada. Só acessa cadastro e login.
Usuário	Pessoa autenticada (com token JWT válido). Dono das próprias categorias e transações.
Agendador	Ator de sistema (tempo). Dispara, todo 5º dia útil, o lançamento automático da renda mensal.

O **Usuário** é uma especialização do **Visitante**: depois de se autenticar, continua podendo usar os casos de uso públicos, mas ganha acesso a todo o restante.

Diagrama (Mermaid)



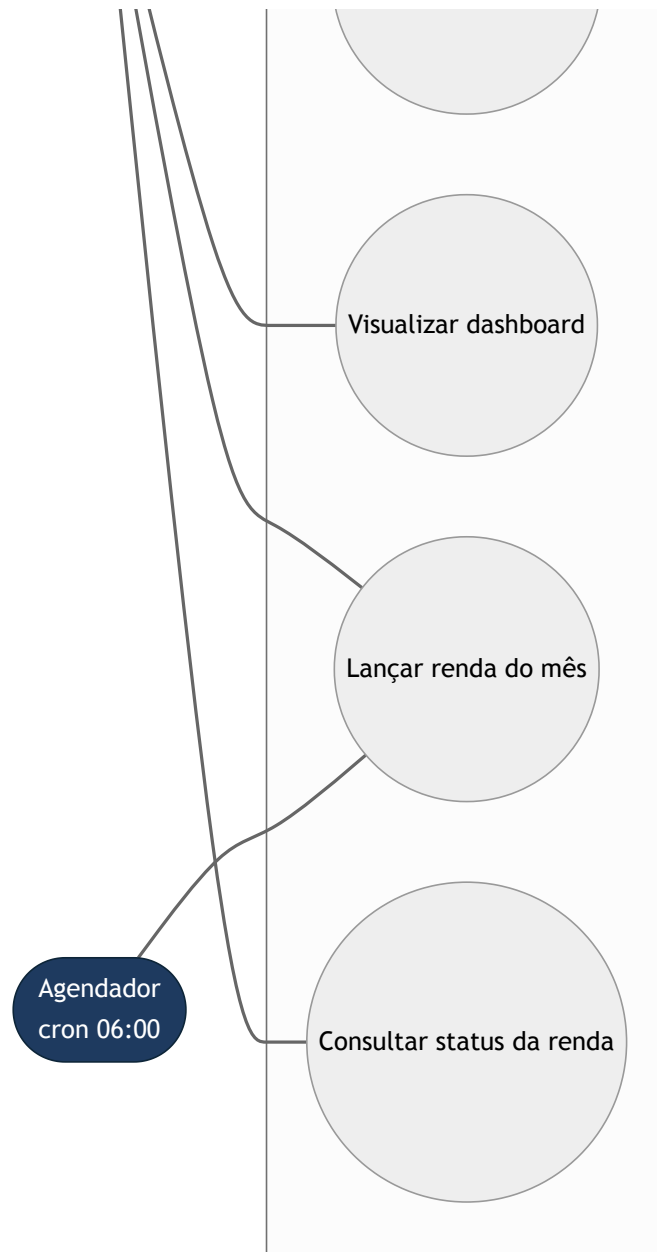


Diagrama (PlantUML — notação UML canônica)

Use este bloco quando precisar gerar a imagem em notação UML formal (elipses e bonecos). Para renderizar, cole o conteúdo em plantuml.com/plantuml ou use a extensão *PlantUML* do VS Code.

```
@startuml
    CasosDeUso
    left to right direction
    skinparam packageStyle rectangle
    skinparam actorStyle awesome

    actor "Visitante" as V
    actor "Usuário" as U
    actor "Agendador" as A << system >>

    U -|> V

    rectangle "Sistema de Controle Financeiro" {
        usecase "Registrar conta" as UC1
        usecase "Fazer login" as UC2
        usecase "Visualizar perfil" as UC3
        usecase "Editar perfil / renda" as UC4
    }
```

```

usecase "Alterar senha" as UC5
usecase "Gerenciar categorias" as UC6
usecase "Gerenciar transações" as UC7
usecase "Visualizar dashboard" as UC8
usecase "Lançar renda do mês" as UC9
usecase "Consultar status da renda" as UC10
}

V --> UC1
V --> UC2
U --> UC3
U --> UC4
U --> UC5
U --> UC6
U --> UC7
U --> UC8
U --> UC9
U --> UC10
A --> UC9

note bottom of UC9
    Editar a renda no perfil (UC4)
    dispara o lançamento (UC9).
end note
@enduml

```

Descrição dos casos de uso

#	Caso de uso	Pré-condição	Fluxo principal
UC1	Registrar conta	E-mail ainda não cadastrado	Informa nome, e-mail, senha e (opcional) renda mensal → sistema cria a conta, gera categorias padrão, lança a renda do mês corrente e devolve o token.
UC2	Fazer login	Conta existente	Informa e-mail e senha → sistema valida (BCrypt) e devolve o token JWT.
UC3	Visualizar perfil	Autenticado	Sistema retorna nome, e-mail e renda mensal do usuário logado.
UC4	Editar perfil / renda	Autenticado	Atualiza nome e renda mensal; ao salvar a renda, o lançamento do mês é reprocessado.
UC5	Alterar senha	Autenticado; senha atual correta	Confere a senha atual e grava o novo hash.
UC6	Gerenciar categorias	Autenticado	Cria, lista, edita e exclui categorias (não permite excluir categoria com transações).
UC7	Gerenciar transações	Autenticado; categoria válida	Cria, lista (paginado), edita e exclui receitas/despesas.
UC8	Visualizar dashboard	Autenticado	Mostra saldo, receitas/despesas do mês, gastos por categoria e fluxo de 6 meses.
UC9	Lançar renda do mês	Renda mensal > 0	Lança a renda como receita no 5º dia útil; pode ser disparado manualmente ou pelo Agendador.
UC10	Consultar status da renda	Autenticado	Informa se a renda do mês já foi lançada e qual a próxima data prevista.

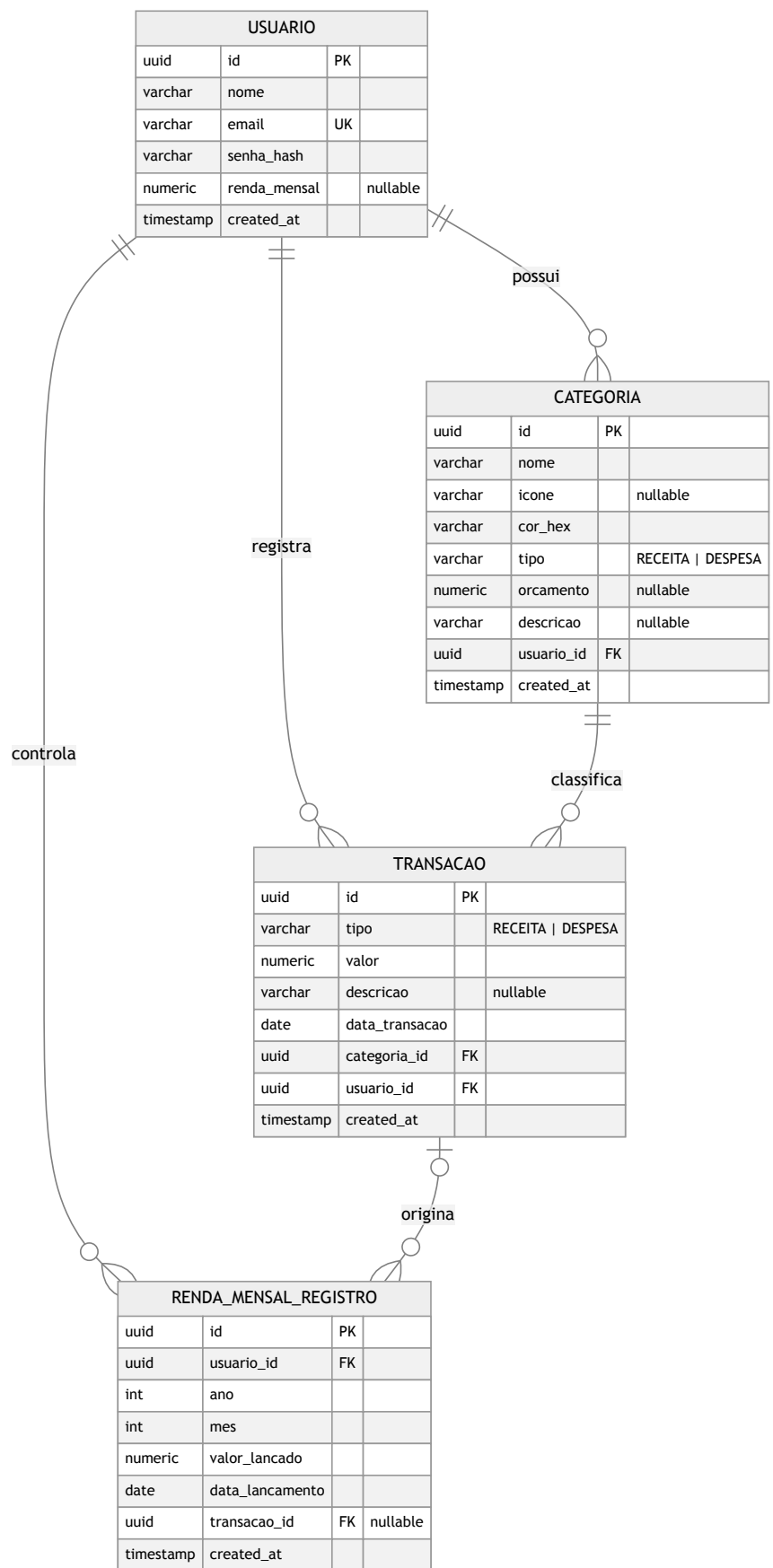
Relações <<include>> e <<extend>>

- **UC4 → UC9** (<<include>> na prática): salvar a renda no perfil aciona o lançamento da renda do mês.
- **UC1 → UC9** (<<include>>): ao registrar a conta com renda informada, o lançamento já é executado.
- Todos os casos de uso do **Usuário** **incluem implicitamente** a validação do token JWT, feita pelo filtro de segurança antes de chegar ao controlador.

Modelo Lógico Relacional

Este documento descreve como o domínio é persistido no **PostgreSQL**. O esquema é gerado pelo Hibernate (`ddl-auto: update`) a partir das entidades JPA; o DDL abaixo é a representação equivalente em SQL.

Diagrama Entidade-Relacionamento



usuario

Coluna	Tipo	Restrições	Descrição
id	UUID	PK	Identificador.
nome	VARCHAR(150)	NOT NULL	Nome do usuário.
email	VARCHAR(255)	NOT NULL, UNIQUE	Login.
senha_hash	VARCHAR(255)	NOT NULL	Senha em hash BCrypt (nunca texto puro).
renda_mensal	NUMERIC(15,2)	NULL	Renda usada no lançamento automático.
created_at	TIMESTAMP	NOT NULL	Data de criação.

categoria

Coluna	Tipo	Restrições	Descrição
id	UUID	PK	Identificador.
nome	VARCHAR(100)	NOT NULL	Nome (único por usuário).
icone	VARCHAR(50)	NULL	Ícone exibido na interface.
cor_hex	VARCHAR(7)	NOT NULL, default #6B7280	Cor em hexadecimal.
tipo	VARCHAR(20)	NOT NULL	RECEITA ou DESPESA .
orcamento	NUMERIC(15,2)	NULL	Limite de gasto planejado.
descricao	VARCHAR(255)	NULL	Texto livre.
usuario_id	UUID	NOT NULL, FK → usuario(id)	Dono da categoria.
created_at	TIMESTAMP	NOT NULL	Data de criação.

transacao

Coluna	Tipo	Restrições	Descrição
id	UUID	PK	Identificador.
tipo	VARCHAR(20)	NOT NULL	RECEITA ou DESPESA .
valor	NUMERIC(15,2)	NOT NULL	Valor do lançamento.
descricao	VARCHAR(255)	NULL	Texto livre.
data_transacao	DATE	NOT NULL	Data do lançamento.
categoria_id	UUID	NOT NULL, FK → categoria(id)	Categoria.
usuario_id	UUID	NOT NULL, FK → usuario(id)	Dono.
created_at	TIMESTAMP	NOT NULL	Data de criação.

renda_mensal_registro

Coluna	Tipo	Restrições	Descrição
id	UUID	PK	Identificador.
usuario_id	UUID	NOT NULL, FK → usuario(id)	Dono do registro.
ano	INTEGER	NOT NULL	Ano de referência.
mes	INTEGER	NOT NULL	Mês de referência (1–12).

Coluna	Tipo	Restrições	Descrição
valor_lancado	NUMERIC(15,2)	NOT NULL	Valor lançado como receita.
data_lancamento	DATE	NOT NULL	Quando o sistema lançou.
transacao_id	UUID	NULL, FK → transacao(id)	Receita gerada (rastreabilidade).
created_at	TIMESTAMP	NOT NULL	Data de criação.
—	—	UNIQUE (usuario_id, ano, mes)	Impede lançamento duplicado no mesmo mês.

Script DDL (PostgreSQL)

```
CREATE TABLE usuario (
  id          UUID PRIMARY KEY,
  nome        VARCHAR(150) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  senha_hash  VARCHAR(255) NOT NULL,
  renda_mensal NUMERIC(15,2),
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE categoria (
  id          UUID PRIMARY KEY,
  nome        VARCHAR(100) NOT NULL,
  icone       VARCHAR(50),
  cor_hex     VARCHAR(7)  NOT NULL DEFAULT '#6B7280',
  tipo        VARCHAR(20) NOT NULL,
  orcamento  NUMERIC(15,2),
  descricao   VARCHAR(255),
  usuario_id  UUID NOT NULL REFERENCES usuario(id),
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE transacao (
  id          UUID PRIMARY KEY,
  tipo        VARCHAR(20)  NOT NULL,
  valor       NUMERIC(15,2) NOT NULL,
  descricao   VARCHAR(255),
  data_transacao DATE NOT NULL,
  categoria_id UUID NOT NULL REFERENCES categoria(id),
  usuario_id  UUID NOT NULL REFERENCES usuario(id),
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE renda_mensal_registro (
  id          UUID PRIMARY KEY,
  usuario_id  UUID NOT NULL REFERENCES usuario(id),
  ano         INTEGER NOT NULL,
  mes         INTEGER NOT NULL,
  valor_lancado NUMERIC(15,2) NOT NULL,
  data_lancamento DATE NOT NULL,
  transacao_id UUID REFERENCES transacao(id),
  created_at  TIMESTAMP NOT NULL DEFAULT now(),
  CONSTRAINT uk_renda_usuario_ano_mes UNIQUE (usuario_id, ano, mes)
);
```

Normalização

O modelo está na **3ª Forma Normal (3FN)**:

- **1FN** — todos os atributos são atômicos; não há grupos repetitivos.
- **2FN** — não há chaves compostas com dependência parcial (as PKs são **UUID** simples).
- **3FN** — não há dependência transitiva: atributos como **cor_hex** ou **valor** dependem apenas da chave primária da sua própria tabela. O **tipo** da transação é redundante em relação ao da categoria por decisão de projeto (permite uma transação avulsa diferir da categoria), e não caracteriza violação.

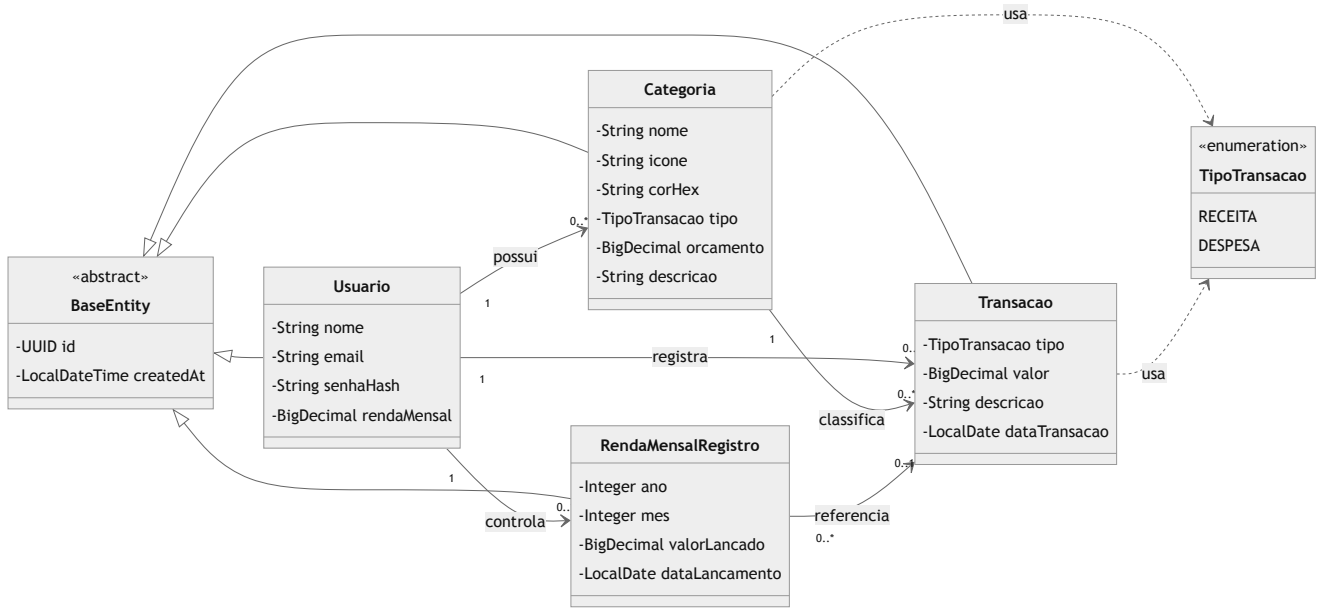
Diagrama de Classes

O sistema segue uma arquitetura em camadas (*Controller* → *Service* → *Repository*). Este documento traz dois recortes complementares:

- 1. **Modelo de domínio** — as entidades persistidas (o que vira tabela no banco).
- 2. **Visão de camadas** — como controladores, serviços e repositórios se relacionam.

1. Modelo de domínio (entidades)

Todas as entidades herdam de `BaseEntity`, que centraliza o identificador (`UUID`) e a data de criação. O tipo de uma categoria ou transação é representado pelo enum `TipoTransacao`.

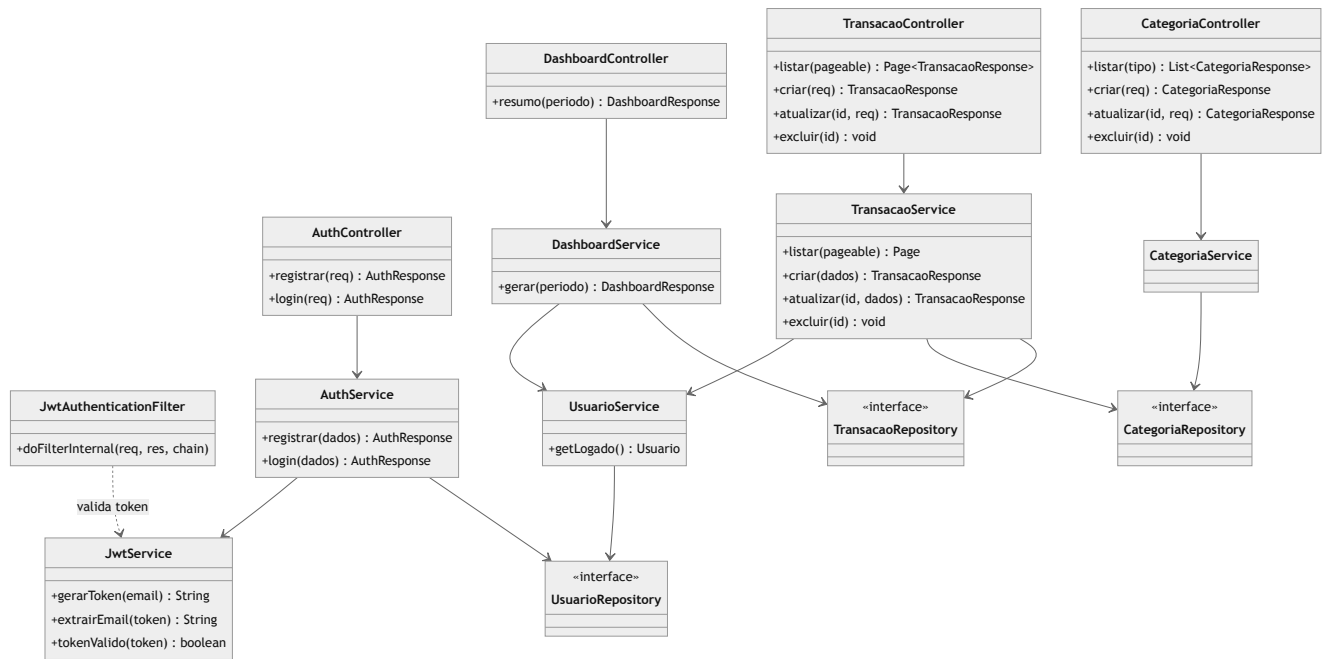


Multiplicidades e regras

Relação	Multiplicidade	Observação
Usuário → Categoria	1 : 0..*	Cada usuário tem suas próprias categorias (criadas por padrão no registro).
Usuário → Transação	1 : 0..*	Toda transação pertence a um usuário.
Categoria → Transação	1 : 0..*	Uma categoria não pode ser excluída se tiver transações.
Usuário → RendaMensalRegistro	1 : 0..*	Um registro por usuário/ano/mês (chave única).
RendaMensalRegistro → Transação	0..* : 0..1	Aponta para a receita gerada, para rastreabilidade.

2. Visão de camadas (arquitetura)

Recorte da arquitetura para uma requisição autenticada. O `JwtAuthenticationFilter` intercepta a requisição antes de chegar ao controlador; os serviços concentram a regra de negócio e usam os repositórios para acessar o banco.



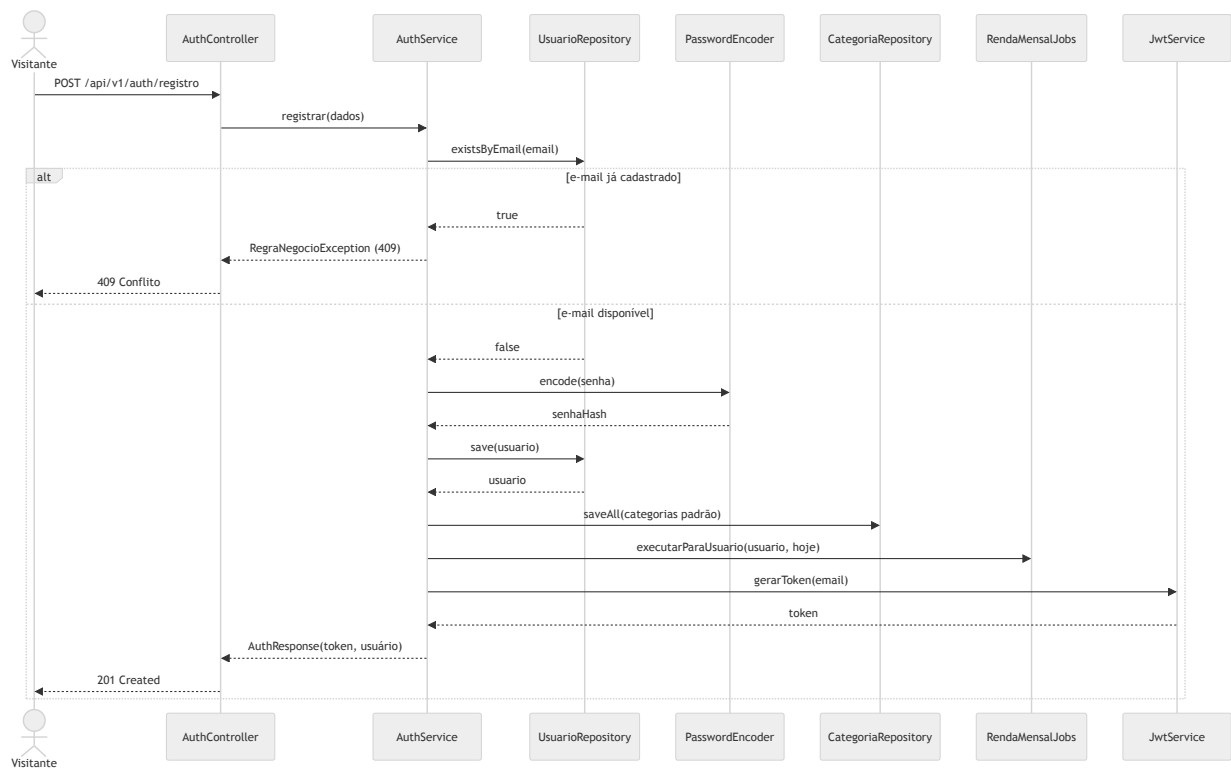
Os repositórios são interfaces Spring Data JPA — não há implementação manual: o Spring gera as consultas a partir do nome dos métodos (ex.: `findByUsuarioIdAndDataTransacaoBetween`).

Diagramas de Sequência

Os diagramas a seguir mostram a troca de mensagens entre os participantes ao longo do tempo. Foram escolhidos quatro fluxos que, juntos, exercitam toda a arquitetura: autenticação, escrita autenticada, leitura agregada e processamento agendado.

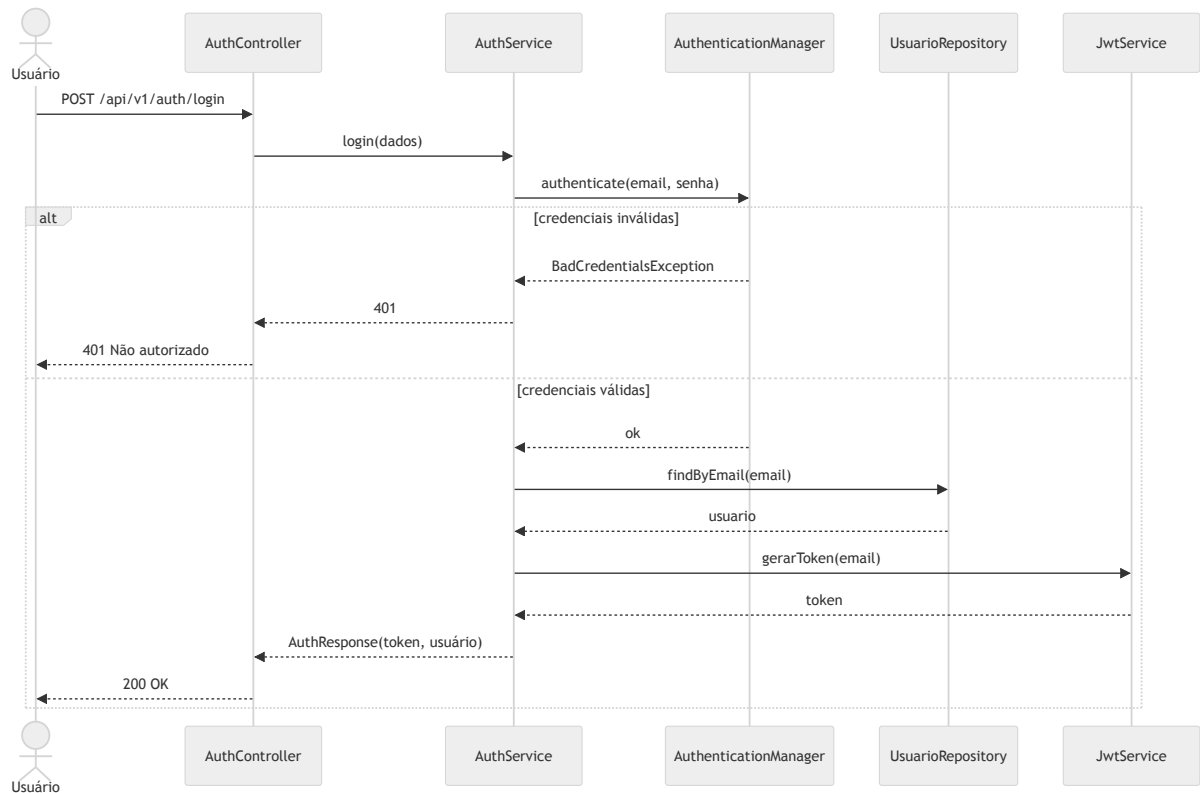
1. Registro de conta

Ao registrar, o sistema valida o e-mail, grava o usuário com senha em hash, cria as categorias padrão, já lança a renda do mês (se informada) e devolve o token.



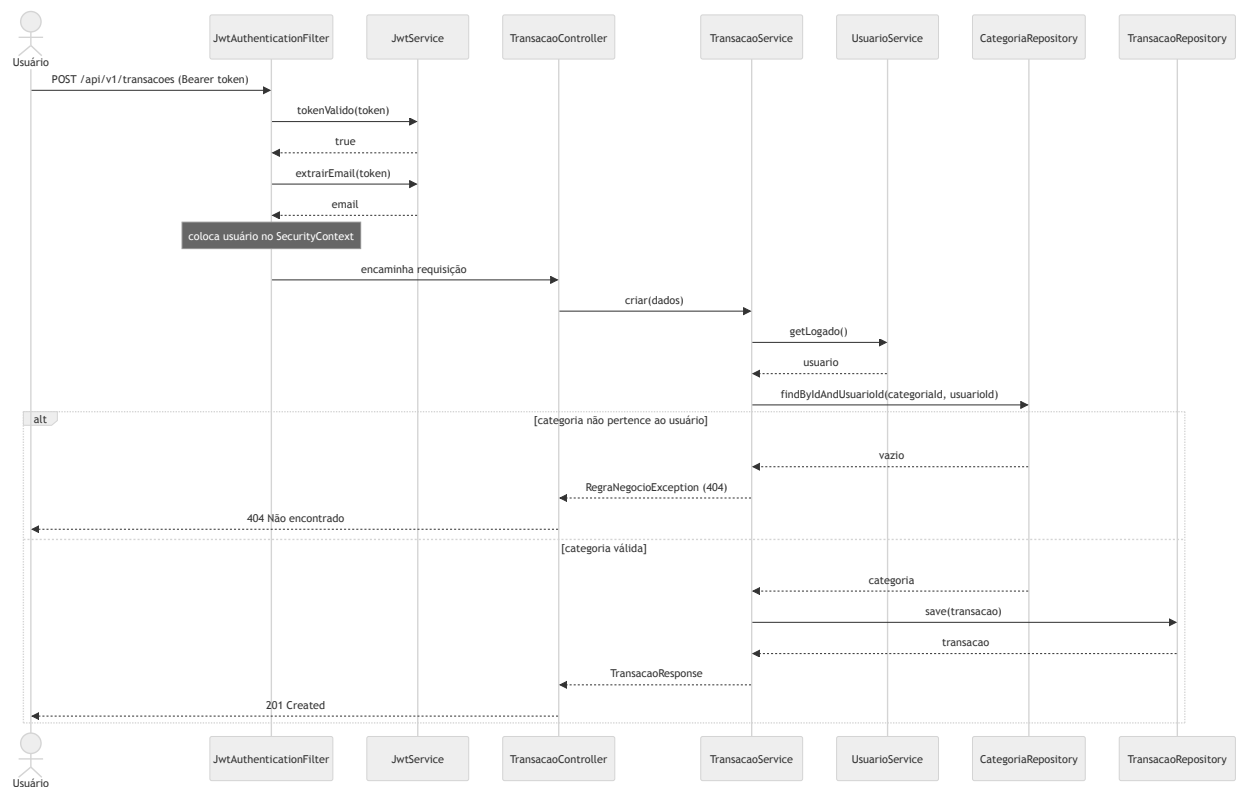
2. Login

A autenticação delega ao `AuthenticationManager` do Spring Security, que compara a senha informada com o hash. Se válida, gera o token JWT.



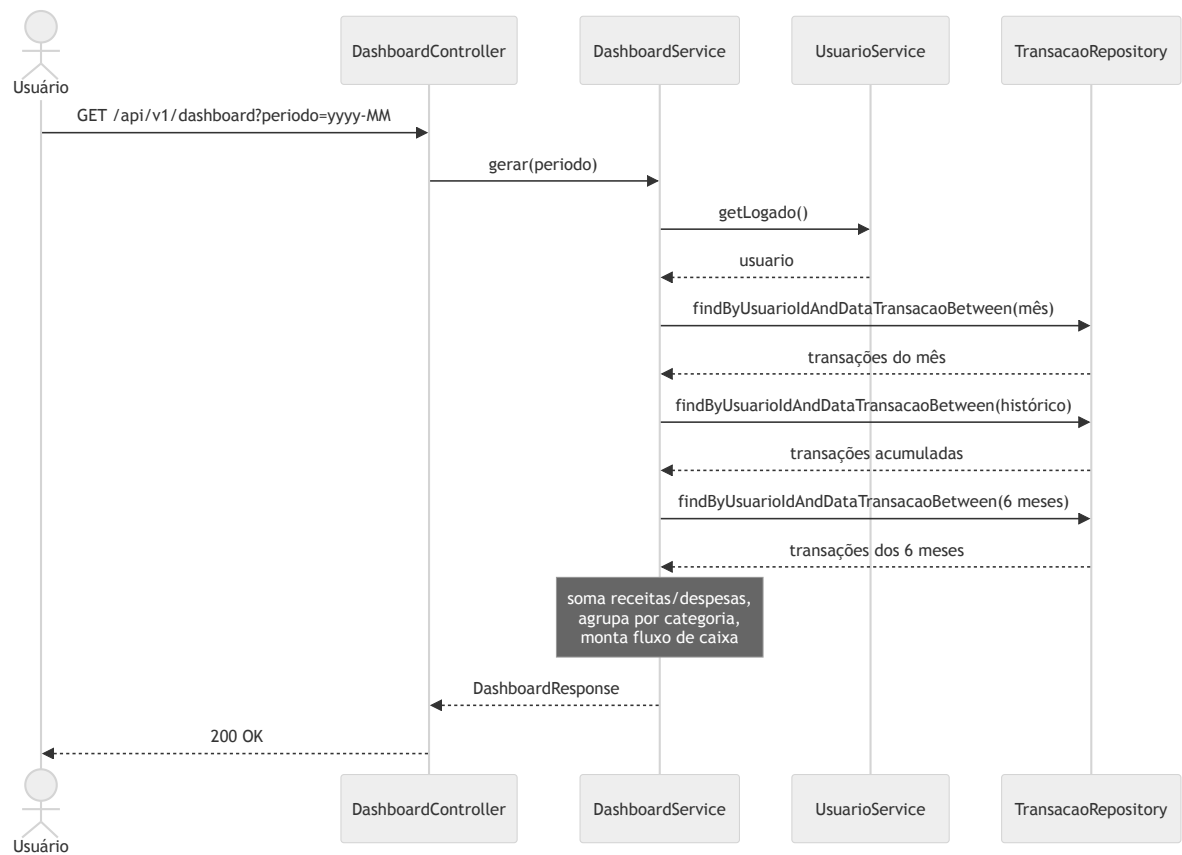
3. Criar transação (requisição autenticada)

Mostra o ciclo completo de uma escrita protegida: o filtro JWT autentica a requisição **antes** do controlador, e o serviço garante que a categoria pertence ao usuário logado.



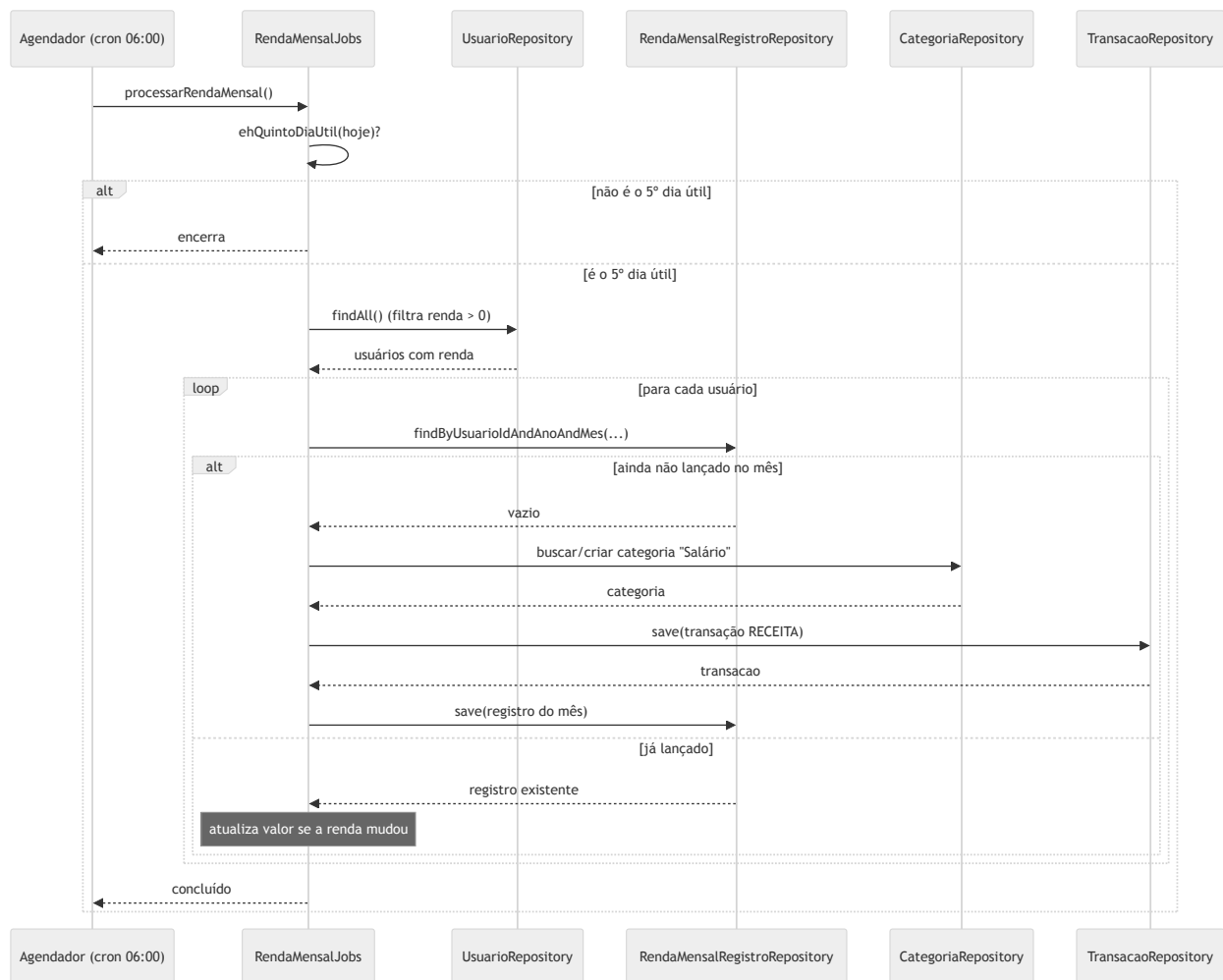
4. Visualizar dashboard (leitura agregada)

O dashboard não tem tabela própria: ele é calculado a partir das transações do usuário no período escolhido.



5. Lançamento automático da renda mensal (job agendado)

Todo dia às 06:00 o agendador dispara a rotina; ela só age se a data for o 5º dia útil do mês. Para cada usuário com renda configurada, lança a receita uma única vez por mês.



O mesmo método `executarParaUsuario(...)` é chamado fora do agendador — no registro da conta, ao salvar a renda no perfil e pelo botão **Lançar renda do mês** — garantindo que o dashboard reflita a renda imediatamente, sem esperar o próximo dia 5.

Arquitetura, Segurança e Guia de Defesa

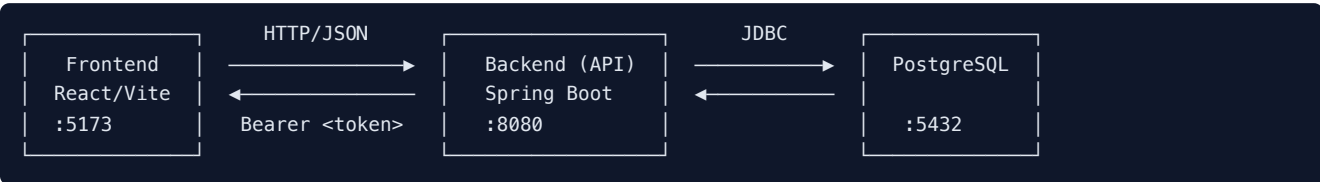
Documento de apoio para o estudo e a apresentação do **Controle Financeiro** à banca. Reúne a visão de arquitetura, o detalhamento da camada de segurança e um roteiro de perguntas e respostas para a defesa ao vivo.

1. Visão geral

O sistema é uma aplicação web de **controle financeiro pessoal**. O usuário cadastra **receitas** e **despesas**, organiza-as em **categorias** e acompanha o resultado em um **dashboard** (saldo, totais do mês, gastos por categoria e fluxo de caixa dos últimos 6 meses). Há ainda um **lançamento automático da renda mensal** no 5º dia útil de cada mês.

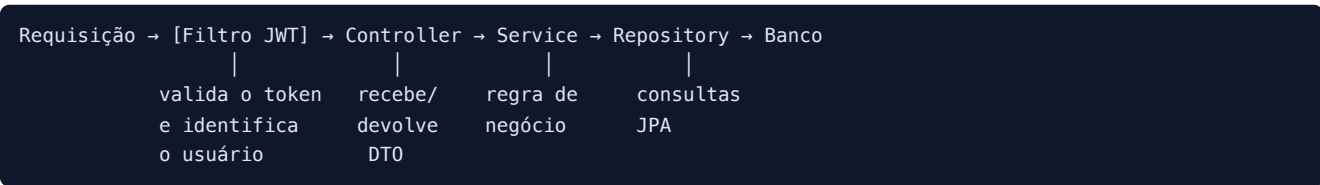
A aplicação é dividida em duas partes independentes:

- **Backend** — API REST em Java/Spring Boot, responsável pela regra de negócio, segurança e persistência.
- **Frontend** — SPA (Single Page Application) em React, que consome a API.



2. Arquitetura do backend (camadas)

O backend segue o padrão clássico em camadas. Cada requisição autenticada percorre o caminho abaixo:



Camada	Pacote	Responsabilidade
Segurança	<code>security/</code>	Valida o token JWT a cada requisição e identifica o usuário.
Controller	<code>controller/</code>	Expõe os endpoints REST; recebe e devolve DTOs (nunca a entidade).
Service	<code>service/</code>	Concentra a regra de negócio; sempre opera no escopo do usuário logado.
Repository	<code>repository/</code>	Interfaces Spring Data JPA — o acesso ao banco.
Entity	<code>entity/</code>	Entidades JPA que viram tabelas.
DTO	<code>dto/</code>	record s de entrada/saída da API.
Job	<code>jobs/</code>	Rotinas agendadas (lançamento da renda, heartbeat).

Por que separar Controller / Service / Repository? Para isolar responsabilidades: o controller cuida do protocolo HTTP, o service da regra de negócio e o repository do banco. Isso facilita teste, manutenção e troca de uma camada sem afetar as outras.

Por que DTO em vez de devolver a entidade? Para não expor dados sensíveis (ex.: o hash da senha do usuário nunca sai da API) e para desacoplar o formato do banco do formato da API.

3. Stack tecnológica e justificativas

Camada	Tecnologia	Por que
Linguagem	Java 21	LTS, tipagem forte, maduro no mercado.
Framework	Spring Boot 3.3	Padrão de mercado para APIs REST; injeção de dependência, segurança e JPA integrados.
Segurança	Spring Security + JWT	Autenticação stateless, sem guardar sessão no servidor.
Persistência	JPA / Hibernate	Mapeia objetos Java para tabelas; gera SQL automaticamente.
Banco	PostgreSQL	Relacional, robusto, open-source.
Boilerplate	Lombok	Gera getters/setters/builders, reduz código repetitivo.
Frontend	React 19 + TypeScript + Vite	SPA reativa, tipada, com build rápido.
UI	PrimeReact + PrimeFlex	Biblioteca de componentes prontos (tabelas, formulários, diálogos).
Estado	Zustand	Gerência de estado simples; guarda o token autenticado.
Gráficos	Chart.js	Pizza (gastos por categoria) e barras (fluxo de caixa).
Formulários	React Hook Form + Zod	Validação declarativa dos formulários.
HTTP	Axios	Cliente HTTP com <i>interceptors</i> para anexar o token.

4. Segurança (ponto central da defesa)

4.1 Autenticação por token JWT

O login **não cria sessão no servidor** — a API é *stateless*. O fluxo:

1. O usuário envia e-mail e senha para `POST /api/v1/auth/login`.
2. O Spring Security confere a senha contra o hash guardado no banco.
3. Se válida, o `JwtService` **gera um token JWT** assinado e o devolve.
4. A cada requisição seguinte, o frontend envia o token no cabeçalho `Authorization: Bearer <token>`.
5. O `JwtAuthenticationFilter` intercepta a requisição, valida o token e marca o usuário como autenticado **apenas para aquela chamada**.

Características do token (implementação real):

Aspecto	Valor
Algoritmo de assinatura	HMAC-SHA (HS256) — <code>Keys.hmacShaKeyFor(secret)</code>
Conteúdo (claims)	<code>subject</code> = e-mail do usuário, <code>issuedAt</code> , <code>expiration</code>
Validade	24 horas (<code>app.jwt.expiration=86400000</code> ms)
Chave secreta	Variável <code>JWT_SECRET</code> (com valor padrão só para desenvolvimento)
Refresh token	Não há — quando expira, o usuário faz login de novo

O que o token NÃO contém: a senha. Ele guarda apenas o e-mail e os horários. Como é assinado com a chave secreta, ninguém consegue forjá-lo sem conhecê-la — qualquer alteração no conteúdo invalida a assinatura.

Validação do token (no filtro): confere a **assinatura** (íntegra e gerada pela nossa chave) e a **expiração** (não vencido). Se qualquer uma falhar, a requisição segue sem autenticação e cai em `401`.

4.2 Senha: hash BCrypt — tem salt?

Sim, tem salt — e é automático. A senha é guardada com `BCryptPasswordEncoder` (Spring Security). O algoritmo bcrypt:

- Gera um **salt aleatório próprio para cada senha** (16 bytes), automaticamente.
- **Embute o salt dentro do próprio hash** — não precisamos guardar o salt em coluna separada.
- Aplica um **fator de custo** (padrão **10**, ou seja, $2^{10} = 1024$ rodadas) que torna a quebra por força bruta lenta.

Formato do que fica salvo na coluna `senha_hash`:

```
$2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
| | |
| | | salt (22 chars) + hash resultante (31 chars)
| | |
| | | fator de custo = 10
| | |
| | | identificador do algoritmo (bcrypt)
```

Consequência prática: dois usuários com a **mesma senha** terão **hashes diferentes**, porque o salt é diferente. Isso impede ataques de *rainbow table* e esconde senhas iguais.

Salt × Pepper: o salt (aleatório, por senha) está presente. **Não** usamos *pepper* (um segredo extra global) nesta versão — é uma melhoria possível de citar como trabalho futuro.

A verificação no login compara a senha digitada com o hash usando `passwordEncoder.matches(...)`, que re-aplica o mesmo salt/custo e confere — **a senha original nunca é desfeita** (hash é via de mão única).

4.3 Autorização (rotas públicas × protegidas)

Configurado em `SecurityConfig`:

- **Públicas** (sem token): `/api/v1/auth/**` (login e registro), `/api/health`, `/api/v1/health`, Swagger e `/error`.
- **Protegidas** (exigem token válido): **todo o resto**.

Além disso, **cada usuário só enxerga os próprios dados**. Os serviços sempre filtram pelo usuário logado — ex.: `transacaoRepository.findByIdAndUsuarioId(id, usuarioId)`. Mesmo que alguém adivinhe o `id` de uma transação de outra pessoa, a consulta não retorna nada (isolamento por usuário).

4.4 CORS

A API libera o frontend (que roda em outra porta) via `CorsConfigurationSource`, permitindo os métodos GET/POST/PUT/PATCH/DELETE. Em produção, o ideal é restringir a origem ao domínio real do frontend.

4.5 Onde o token fica no frontend

O token é guardado no **localStorage** do navegador (chave `glass-finance-auth`, persistido pelo Zustand). O *interceptor* do Axios lê esse token e o injeta no cabeçalho de toda requisição. Se a API responder **401**, o frontend limpa o storage e redireciona para o login.

Ponto de honestidade para a banca: guardar JWT em `localStorage` é simples e comum, mas fica exposto a ataques XSS. Alternativa mais segura: cookie `httpOnly`. Citar isso mostra domínio do assunto.

5. Frontend (organização)

SPA em React com TypeScript. Principais pastas em `frontend/src`:

Pasta	Conteúdo
<code>pages/</code>	Telas: Login, Registro, Painei (dashboard), Transações, Categorias, Perfil.
<code>services/</code>	Funções que chamam a API (um arquivo por recurso).
<code>store/</code>	Estado global com Zustand (<code>autenticacao.ts</code> guarda usuário e token).
<code>components/common/</code>	Componentes reutilizáveis (cartões, gráficos, diálogos, rota privada).
<code>types/</code>	Interfaces TypeScript espelhando os DTOs da API.
<code>assets/styles/</code>	Tema "glass" (vidro), variáveis CSS e animações.

Proteção de rotas: o componente `RotaPrivada` impede o acesso às telas internas sem autenticação. Ao abrir o app, o `checkAuth()` valida o token chamando `/usuarios/perfil`; se falhar, faz logout.

Resiliência (cold start do Render): o Axios tem `timeout` de 90 s e mensagens amigáveis, porque no plano gratuito do Render o backend pode estar "dormindo" e leva alguns segundos para acordar.

6. Integrações com APIs externas

O sistema não consome nenhuma API de terceiros. Não há gateway de pagamento, Open Finance, envio de e-mail ou serviço externo. Toda a comunicação é **frontend ↔ backend próprio**.

O único elemento "externo" é a **hospedagem no Render** (plano gratuito), que pode suspender o serviço após inatividade. Para isso existe:

- O endpoint público e leve `GET /api/health`, usado para verificar/acordar a API.
- A job `MonitoramentoJobs`, que registra um *heartbeat* no log a cada 2 minutos enquanto a aplicação está ativa.

Se a banca perguntar sobre "APIs externas", a resposta correta é: **não há integrações externas**; a API consumida pelo frontend é a do próprio projeto. Esse é um recorte proposital da versão simplificada.

7. Regras de negócio que valem destacar

1. **Categorias padrão no cadastro** — ao criar a conta, o sistema já insere categorias (Salário, Alimentação, Transporte...) para o usuário não começar com a tela vazia.
2. **Dashboard é calculado, não armazenado** — não existe tabela de "resumo"; os números são somados a partir das transações no momento da consulta.
3. **Lançamento automático da renda mensal** — todo 5º dia útil, a renda configurada no perfil vira uma receita "Salário". Um registro por mês (`UNIQUE(usuario_id, ano, mes)`) evita duplicidade. A mesma rotina roda ao cadastrar a conta, ao salvar a renda no perfil e por um botão manual, atualizando o dashboard na hora.
4. **Integridade ao excluir** — uma categoria com transações **não** pode ser excluída (evita transações órfãs).

8. Roteiro de defesa — perguntas prováveis da banca

A banca pode pedir alterações ao vivo (prevenção contra uso indevido de IA). Estude estas respostas e saiba **onde**

mexer no código.

Pergunta	Resposta curta	Onde está no código
Como funciona o login?	Valida senha com BCrypt e devolve um JWT assinado (HS256, 24h).	<code>AuthService.login</code> , <code>JwtService</code>
A senha é guardada como?	Hash bcrypt com salt aleatório embutido (custo 10); nunca em texto puro.	<code>Usuario.senhaHash</code> , <code>SecurityConfig.passwordEncoder</code>
O hash tem salt?	Sim, o bcrypt gera um salt único por senha automaticamente.	<code>BCryptPasswordEncoder</code>
Como o token é validado a cada requisição?	Um filtro lê o <code>Bearer</code> , confere assinatura e expiração e identifica o usuário.	<code>JwtAuthenticationFilter</code>
Um usuário vê dados de outro?	Não; toda consulta filtra pelo usuário logado.	<code>*Service.buscarEntidade</code> , <code>findBy...AndUsuarioId</code>
O que é stateless?	O servidor não guarda sessão; cada requisição se identifica pelo token.	<code>SecurityConfig</code> (<code>SessionCreationPolicy.STATELESS</code>)
Como o dashboard é montado?	Somando as transações do período direto do banco.	<code>DashboardService.gerar</code>
Como criar um novo campo/endpoint?	Entidade → DTO → Repository → Service → Controller.	seguir o fluxo das camadas
Por que UUID e não id sequencial?	Difícil de adivinhar (não dá para enumerar registros) e bom para sistemas distribuídos.	<code>BaseEntity</code>

Dica de apresentação: mostre o sistema rodando (login → lançar uma transação → ver o dashboard mudar → categorias → perfil). Use dados fictícios rápidos. Deixe ~70% do tempo para a demonstração e o resto para explicar arquitetura/segurança.

Segurança — Documento Técnico Detalhado

Documento aprofundado da camada de segurança do **Controle Financeiro**, preparado para defesa diante de banca especializada. Cada mecanismo é descrito com o **arquivo**, o **método**, o **algoritmo** e o **porquê** da decisão.

Todos os caminhos abaixo partem de `backend/src/main/java/com/financas`.

1. Panorama: o que protege o quê

Camada	Componente	Protege contra
Armazenamento de senha	<code>BCryptPasswordEncoder</code>	Vazamento de senhas se o banco for comprometido.
Autenticação	<code>AuthenticationManager</code> + <code>UsuarioDetailsService</code>	Login com credenciais inválidas.
Emissão de identidade	<code>JwtService.gerarToken</code>	Necessidade de re-enviar senha a cada requisição.
Verificação por requisição	<code>JwtAuthenticationFilter</code>	Acesso sem token / com token forjado ou expirado.
Autorização	<code>SecurityConfig</code> (filter chain)	Acesso a endpoints protegidos sem autenticação.
Isolamento de dados	Consultas <code>...AndUsuarioId(...)</code>	Um usuário acessar dados de outro (IDOR).
Transporte	CORS + (HTTPS em produção)	Origem não autorizada; interceptação.

2. Senha: hash, salt e algoritmo

2.1 Onde o hash é gerado

Arquivo: `config/SecurityConfig.java`

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Esse bean é injetado em quem precisa lidar com senha:

- `AuthService.registrar` → `passwordEncoder.encode(dados.senha())` (linha 49) — ao criar a conta.
- `UsuarioService.alterarSenha` → `passwordEncoder.encode(dados.novaSenha())` (linha 61) — ao trocar a senha.
- `UsuarioService.alterarSenha` → `passwordEncoder.matches(...)` (linha 57) — confere a senha atual.

2.2 Algoritmo: bcrypt

A senha **nunca** é guardada em texto puro. Usamos **bcrypt**, um algoritmo de hash projetado especificamente para senhas (família *Blowfish*). Características:

- Função de mão única (one-way):** dado o hash, não há como voltar à senha original.

- **Lento de propósito (key stretching):** aplica um **fator de custo** (padrão **10** → $2^{10} = 1024$ iterações). Isso torna a força bruta cara, mesmo com GPU.
- **Salt embutido e automático:** ver abaixo.

2.3 Onde o salt entra — "o valor aleatório na frente do hash"

Você não chama o salt manualmente — o `bcrypt` gera e gerencia. A cada `encode(...)`, o `bcrypt`:

1. Sorteia um **salt aleatório de 16 bytes** (via `SecureRandom`).
2. Combina salt + senha e roda as 1024 iterações.
3. **Grava o salt dentro da própria string do hash**, logo "na frente" do resultado.

Formato armazenado na coluna `usuario.senha_hash`:

```
$2a$10$Q9mZ3kP1nW8vY2cR4tU6e8XyZ...restante
```

Diagrama explicando a estrutura do hash:

- `$2a$`: versão do algoritmo (bcrypt, \$2a\$)
- `10`: fator de custo = 10
- `$Q9mZ3kP1nW8vY2cR4tU6e8XyZ`: SALT aleatório (22 chars Base64) ← "o valor da frente"
- `...restante`: hash final (31 chars)

Por isso o hash muda toda vez: se você cadastrar dois usuários com a senha `123456`, os hashes serão **diferentes**, porque o salt é diferente em cada um. Demonstre isso na banca cadastrando dois usuários com a mesma senha e olhando a tabela `usuario`.

Salt × Pepper:

- **Salt (presente):** aleatório, **único por senha**, guardado junto ao hash. Derrota *rainbow tables* e esconde senhas iguais.
- **Pepper (ausente nesta versão):** seria um segredo **global** somado à senha antes do hash, guardado **fora** do banco (ex.: variável de ambiente). É uma melhoria possível — citar como trabalho futuro.

2.4 Como o sistema confere se a senha está correta

No `bcrypt` **não se descriptografa**. A verificação reaplica o mesmo processo:

1. `matches(senhaDigitada, hashGuardado)` lê o **salt** e o **custo** de dentro do `hashGuardado`.
2. Reprocessa a `senhaDigitada` com esse mesmo salt/custo.
3. Compara o resultado com o hash guardado. Igual → senha correta.

No **login**, isso acontece dentro do `AuthenticationManager` (Spring Security), que usa o `UsuarioDetailsService` para carregar o hash e o `BCryptPasswordEncoder` para comparar — ver seção 4.

3. Token JWT: o que é, onde nasce, o que carrega

Arquivo: `security/JwtService.java`

3.1 O que é um JWT

Um **JSON Web Token** tem três partes separadas por ponto: `header.payload.signature`.

- **Header:** algoritmo da assinatura (aqui, **HS256** = HMAC-SHA256).
- **Payload (claims):** dados não secretos. No nosso caso: `sub` (e-mail), `iat` (emitido em), `exp` (expira em).
- **Signature:** HMAC-SHA256 de `header.payload` usando a **chave secreta** do servidor.

O payload **não é criptografado**, só assinado. Por isso **não guardamos senha nem dados sensíveis** ali — apenas o e-mail. A assinatura garante **integridade** (ninguém altera o conteúdo sem invalidar o token), não sigilo.

3.2 Onde o token é criado

JwtService.gerarToken(String email) — linhas 33-42:

```
public String gerarToken(String email) {
    Date agora = new Date();
    Date expira = new Date(agora.getTime() + expiracaoMs); // +24h
    return Jwts.builder()
        .subject(email)           // sub = e-mail do usuário
        .issuedAt(agora)          // iat
        .expiration(expira)       // exp
        .signWith(chave)          // assina com HMAC-SHA (HS256)
        .compact();
}
```

Quem chama: `AuthService.montarResposta` (linha 73), tanto no **registro** quanto no **login**. O token volta no corpo da resposta (`AuthResponse.accessToken`).

3.3 A chave secreta e o algoritmo

Construtor do `JwtService` (linhas 25-30):

```
this.chave = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
```

- `secret` vem de `app.jwt.secret` (variável `JWT_SECRET`, com fallback só para desenvolvimento).
- `Keys.hmacShaKeyFor(...)` cria uma chave **HMAC-SHA**. Como a chave do nosso secret tem ≥ 256 bits, a biblioteca (JJWT) assina em **HS256**.
- **HMAC** = *Hash-based Message Authentication Code*: combina a mensagem com a chave secreta e aplica SHA-256. Só quem tem a chave consegue gerar/validar a assinatura.

3.4 Validade

`app.jwt.expiration=86400000` ms = **24 horas**. Não há *refresh token*: ao expirar, o usuário faz login de novo. (Trade-off consciente: simplicidade × conveniência.)

4. Fluxo de LOGIN (passo a passo, com algoritmos)

Arquivos: `controller/AuthController.java` → `service/AuthService.java`

```

POST /api/v1/auth/login    { email, senha }
    |
    ▼
AuthController.login(dados)                               AuthController.java
    |
    ▼
AuthService.login(dados)                                   AuthService.java:61
    |
    └─ authenticationManager.authenticate(                linha 63
        new UsernamePasswordAuthenticationToken(email, senha))
        |
        └─ chama UsuarioDetailsService.loadUserByUsername(email)
            └─ → carrega Usuario e devolve User(email, senhaHash, [])  UsuarioDetailsService.java:25

```

```

└─ DaoAuthProvider usa BCryptPasswordEncoder.matches(
    senhaDigitada, senhaHash)
    • senha errada → lança BadCredentialsException

└─ (se passou) usuarioRepository.findByEmail(email)          linha 66
└─ montarResposta(usuario) → jwtService.gerarToken(email)   linha 73
    ↓
    AuthResponse { accessToken, tokenType="Bearer", expiresIn, usuario }

```

Senha errada: o `AuthenticationManager` lança `BadCredentialsException`, capturada pelo `GlobalExceptionHandler` (linha 29-31), que devolve **HTTP 401** com a mensagem genérica *"E-mail ou senha inválidos"* (mensagem genérica de propósito, para não revelar se o e-mail existe).

5. Fluxo de toda REQUISIÇÃO autenticada (o "filter chain")

Arquivos: `config/SecurityConfig.java` + `security/JwtAuthenticationFilter.java`

5.1 Configuração do filtro (quem é público × protegido)

`SecurityConfig.securityFilterChain` define:

- **Stateless:** `SessionCreationPolicy.STATELESS` — o servidor **não guarda sessão**; cada requisição se identifica sozinha pelo token.
- **Públicos:** `/api/v1/auth/**`, `/api/health`, `/api/v1/health`, Swagger, `/error`.
- **Protegidos:** `anyRequest().authenticated()` — todo o resto exige autenticação.
- O `JwtAuthenticationFilter` é inserido **antes** do `UsernamePasswordAuthenticationFilter`:

```
.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
```

- **Resposta a não autenticado = 401:** configuramos um `authenticationEntryPoint` que devolve **401 Unauthorized** quando falta token (em vez do **403** padrão do Spring). Assim distinguimos *não autenticado* (401) de *autenticado sem permissão* (403), como manda o HTTP — e isso é verificado pelo teste de integração (ver `docs/TESTES-AUTOMATIZADOS.md`).

5.2 O filtro, linha a linha — `JwtAuthenticationFilter.doFilterInternal`

```

String header = request.getHeader("Authorization");          // linha 37
if (header != null && header.startsWith("Bearer ")) {        // linha 39
    String token = header.substring(7);                       // linha 40 (tira "Bearer ")
    if (jwtService.tokenValido(token)                         // linha 42 (assinatura + expiração)
        && SecurityContextHolder.getContext().getAuthentication() == null) {
        String email = jwtService.extrairEmail(token);        // linha 45 (lê o "sub")
        UserDetails usuario =
            usuarioDetailsService.loadUserByUsername(email);   // linha 46 (carrega do banco)
        var auth = new UsernamePasswordAuthenticationToken(
            usuario, null, usuario.getAuthorities());          // linha 48-49 (authorities = papéis)
        auth.setDetails(...);                                  // linha 50
        SecurityContextHolder.getContext()
            .setAuthentication(auth);                           // linha 51 (marca como autenticado)
    }
}
filterChain.doFilter(request, response);                      // linha 55 (segue o fluxo)

```

5.3 Como o token é validado — `JwtService.tokenValido / lerClaims`

```

public boolean tokenValido(String token) {                                // linha 50
    try {
        return lerClaims(token).getExpiration().after(new Date()); // não expirou?
    } catch (Exception e) {
        return false;                                                // assinatura inválida/malformado → false
    }
}

private Claims lerClaims(String token) {                                // linha 58
    return Jwts.parser()
        .verifyWith(chave)      // confere a assinatura HMAC com a NOSSA chave
        .build()
        .parseSignedClaims(token) // lança exceção se a assinatura não bater
        .getPayload();
}

```

Ou seja, a validação cobre **duas coisas**: (1) **assinatura** — o token foi assinado pela nossa chave e não foi adulterado; (2) **expiração** — ainda está dentro das 24h. Qualquer falha → o filtro **não autentica** e a requisição cai em **401** ao tentar acessar rota protegida.

5.4 Tipo de permissão do usuário (authorities)

Hoje, `UsuarioDetailsService.loadUserByUsername` cria o usuário com **lista de papéis vazia** (`List.of()`, linha 30). Ou seja, **não há perfis diferenciados** (admin/comum) nesta versão: a autorização é binária — *autenticado* × *não autenticado* — e o isolamento por usuário (seção 6) faz o resto.

Pergunta provável da banca: *"E se quisesse um admin?"* — Resposta: bastaria popular as authorities (ex.: `List.of(new SimpleGrantedAuthority("ROLE_ADMIN"))`) e proteger endpoints com `@PreAuthorize("hasRole('ADMIN')")`. A estrutura já suporta; só não foi necessário no escopo.

6. Autorização de dados — isolamento por usuário (anti-IDOR)

Mesmo autenticado, o usuário só acessa o que é dele. Isso é garantido **na consulta**, não só na URL:

- `UsuarioService.getLogado()` (linha 35-38) lê o e-mail do `SecurityContext` e busca o `Usuario` atual.
- As consultas sempre amarram o `usuarioId`:
 - `transacaoRepository.findByIdAndUsuarioId(id, usuarioId)`
 - `categoriaRepository.findByIdAndUsuarioId(id, usuarioId)`

Efeito: se o usuário A tentar `GET /api/v1/transacoes/{id}` com o `id` de uma transação do usuário B, a consulta retorna vazio → **404**, não os dados de B. Isso previne **IDOR** (*Insecure Direct Object Reference*), uma falha clássica que a banca pode testar.

7. Outras decisões e pontos de honestidade

Tema	Situação atual	Observação para a banca
Transporte	HTTP em dev	Em produção, HTTPS/TLS obrigatório (o token viaja no header).
Armazenamento do token no front	<code>localStorage</code>	Simples, porém exposto a XSS . Alternativa: cookie <code>httpOnly</code> .
CORS	<code>allowedOriginPatterns("*")</code>	Em produção, restringir ao domínio real do frontend.

Tema	Situação atual	Observação para a banca
Segredo JWT	Variável <code>JWT_SECRET</code>	O fallback do código é só para dev; em produção, segredo forte e fora do código.
Revogação de token	Não há lista de revogação	Como é stateless, um token válido vale até expirar. Mitigação: expiração curta / blacklist (trabalho futuro).
Força da senha	Validação básica no DTO	Pode-se reforçar política de senha (tamanho, complexidade).
Proteção de força bruta no login	Não há rate limit	Citar como melhoria (ex.: bloqueio após N tentativas).

Demonstrar consciência desses limites **conta a favor** na defesa — mostra que você entende segurança, não só implementou.

8. Resumo de algoritmos e bibliotecas

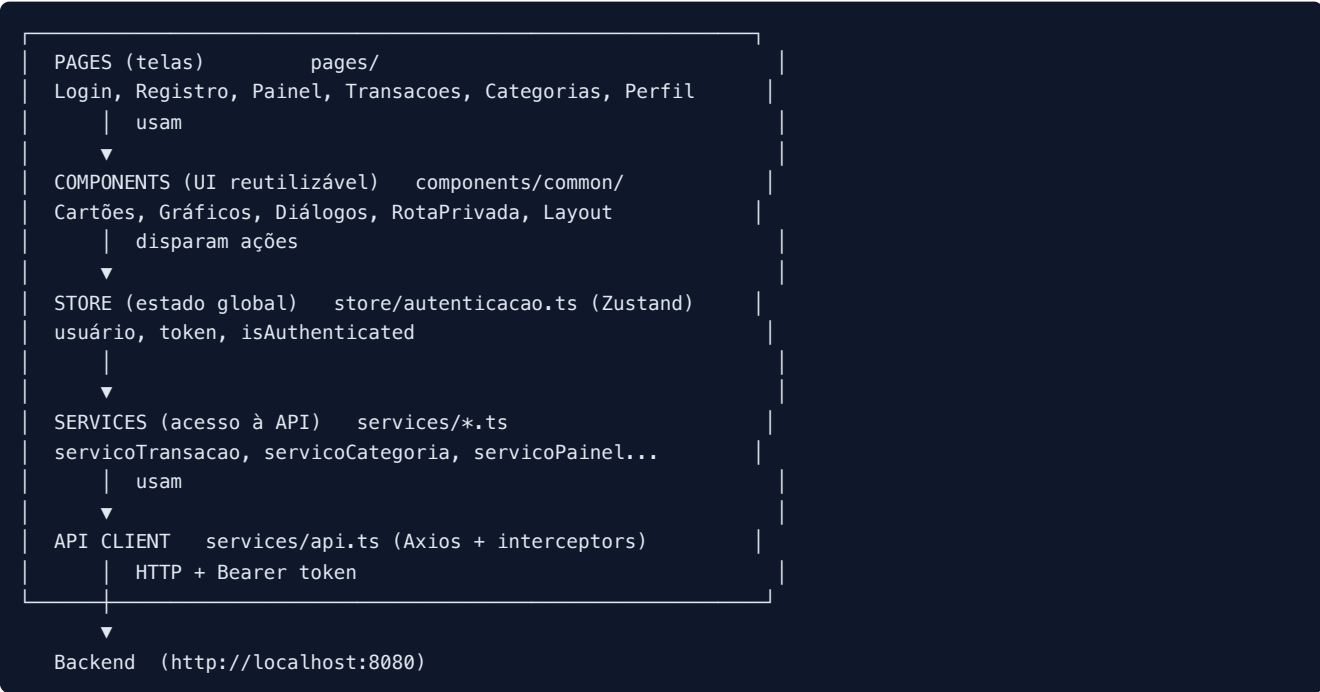
Função	Algoritmo / Tecnologia	Biblioteca
Hash de senha	<code>bcrypt</code> (custo 10, salt aleatório)	Spring Security Crypto
Assinatura do token	<code>HMAC-SHA256 (HS256)</code>	JJWT (<code>io.jsonwebtoken</code> 0.12.5)
Geração de salt	<code>SecureRandom</code> (dentro do <code>bcrypt</code>)	Spring Security Crypto
Identidade/sessão	<code>JWT stateless</code>	JJWT + Spring Security
Controle de acesso	Filter chain + <code>SecurityContext</code>	Spring Security 6

Arquitetura do Frontend

Como a interface web é organizada, em camadas, e como cada parte se conecta com a API. Caminhos a partir de `frontend/src`.

1. Visão em camadas

O frontend é uma **SPA** (Single Page Application): uma única página HTML que troca de tela sem recarregar, conversando com a API por JSON.



Camada	Pasta	Responsabilidade
Pages	<code>pages/</code>	As telas do sistema (uma por rota).
Components	<code>components/common/</code>	Peças de UI reaproveitadas (cartões, gráficos, diálogos, layout, rota privada).
Store	<code>store/</code>	Estado global com Zustand — guarda usuário e token, persistidos no <code>localStorage</code> .
Services	<code>services/</code>	Funções que chamam a API (uma por recurso). Isolam o componente da URL/HTTP.
API client	<code>services/api.ts</code>	Instância única do Axios com <i>interceptors</i> que anexam o token e tratam erros/401.
Types	<code>types/</code>	Interfaces TypeScript que espelham os DTOs da API (tipagem ponta a ponta).
Hooks	<code>hooks/</code>	Lógica reutilizável (ex.: notificação/toast).
Styles	<code>assets/styles/</code>	Tema visual "glass" (vidro), variáveis e animações CSS.

Por que separar `services` das `pages`? Para que a tela não saiba *como* a API é chamada — ela só pede `transacaoService.create(...)`. Se a URL ou o formato mudar, muda-se em um lugar só. É o mesmo princípio do backend (Controller × Service).

2. Roteamento e proteção de telas

Arquivo: `App.tsx`

- Usa **React Router**. Rotas públicas: `/login` e `/registro`. As demais ficam dentro de `<PrivateRoute>`.

- `RotaPrivada.tsx` (`components/common`) decide:
 - ainda verificando token → mostra tela de carregamento;
 - não autenticado → redireciona para `/login`;
 - autenticado → libera a tela (`<Outlet/>`).
- **Code splitting:** cada página é carregada sob demanda (`lazy(() => import(...))`), deixando o carregamento inicial mais leve.
- Ao abrir o app, um `useEffect` chama `checkAuth()`: se houver token salvo, confirma no backend (`GET /usuarios/perfil`) se ainda é válido.

3. Autenticação no front (o ciclo do token)

Arquivo: `store/autenticacao.ts` (Zustand com `persist`)

1. **Login/registro:** `authStore.login()` chama `POST /api/v1/auth/login` , recebe `accessToken` e guarda no estado.
2. **Persistência:** o middleware `persist` salva `token` e `user` no `localStorage` (chave `glass-finance-auth`). Assim, ao recarregar a página, o usuário continua logado.
3. **Envio automático:** o *interceptor de request* do Axios (`services/api.ts`) lê o token do `localStorage` e injeta `Authorization: Bearer <token>` em toda requisição.
4. **Expiração/erro:** o *interceptor de response* detecta **401**, limpa o storage e redireciona para `/login` .
5. **Logout:** remove o token do estado e do header do Axios.

```

Login → token no Zustand → persistido no localStorage
      |
cada requisição → interceptor Axios lê o token → header Bearer
      |
resposta 401 → limpa storage → volta ao /login
  
```

4. Fluxo completo de uma ação (ex.: lançar um gasto)

```

Tela Transacoes (pages/Transacoes.tsx)
| usuário preenche o formulário (React Hook Form + Zod valida)
▼
transacaoService.create(dados)          (services/servicoTransacao.ts)
|
▼
api.post('/api/v1/transacoes', dados) (services/api.ts)
| interceptor anexa Authorization: Bearer <token>
▼
Backend valida o token e grava → devolve a transação criada
|
▼
Tela atualiza a lista e mostra um toast de sucesso
  
```

5. Bibliotecas principais e o papel de cada uma

Biblioteca	Papel
React 19 + TypeScript	Base da SPA, com tipagem estática.
Vite	Servidor de desenvolvimento e <i>build</i> de produção (rápido).
React Router	Navegação entre telas sem recarregar a página.
Zustand	Estado global simples (usuário/token), com persistência.
Axios	Cliente HTTP com <i>interceptors</i> (token e tratamento de 401).

Biblioteca	Papel
React Hook Form + Zod	Formulários e validação declarativa dos campos.
PrimeReact + PrimeFlex	Componentes de UI (tabelas, inputs, diálogos) e layout.
Chart.js	Gráficos do dashboard (pizza de gastos, barras de fluxo).
lucide-react / primeicons	Ícones.

6. Resiliência a *cold start* (deploy gratuito)

O Axios usa `timeout: 90000` (90s) e mensagens amigáveis quando o backend está "acordando" no Render gratuito. Se o servidor demora, o usuário vê "O servidor pode estar iniciando..." em vez de um erro seco.

7. Pontos para a banca

- **Onde fica o token?** No `localStorage` (via Zustand `persist`). É simples; a alternativa mais segura contra XSS seria cookie `httpOnly`.
- **Quem protege as telas?** O componente `RotaPrivada` no front (UX) e o backend (segurança real). O front nunca é a única defesa — o backend valida o token em toda requisição.
- **Como o token chega na API?** Sempre pelo *interceptor* do Axios, nunca manualmente em cada chamada.

Testes Automatizados (JUnit)

O projeto tem testes automatizados que provam, sem intervenção manual, que a segurança e o fluxo principal funcionam. São de dois tipos: **unitário** e **integração**.

Como rodar

```
cd backend
mvn test
```

Os testes usam um banco **H2 em memória** (configurado em `backend/src/test/resources/application.properties`), então **não é preciso ter o PostgreSQL rodando**. Ao final, o Maven mostra algo como:

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

O que cada teste prova

1. Teste UNITÁRIO — JwtServiceTest

Arquivo: `backend/src/test/java/com/financas/security/JwtServiceTest.java`

Testa o `JwtService` isolado (sem subir o Spring, sem banco). Casos:

Teste	O que garante
<code>gerarToken_eExtrairEmail</code>	O e-mail colocado no token é o mesmo que se lê de volta.
<code>tokenRecemCriado_eValido</code>	Token dentro da validade é aceito.
<code>tokenExpirado_eInvalido</code>	Token vencido é rejeitado.
<code>tokenComOutraChave_eInvalido</code>	Token assinado com outra chave é rejeitado (anti-falsificação).
<code>textoQualquer_naoEValido</code>	Texto que não é um JWT é tratado como inválido, sem quebrar.

O caso `tokenComOutraChave_eInvalido` é ótimo para a banca de segurança: prova na prática que, sem a chave secreta, **não dá para forjar um token válido**.

2. Teste de INTEGRAÇÃO — AuthFlowIntegrationTest

Arquivo: `backend/src/test/java/com/financas/integracao/AuthFlowIntegrationTest.java`

Sobe a aplicação inteira (com H2) e exercita o caminho real via `MockMvc`. Casos:

Teste	O que garante
<code>fluxoCompletoDeAutenticacao</code>	Registrar (201) → Login (200, devolve token) → acessar <code>/usuarios/perfil</code> com token (200) → sem token (401).
<code>loginComSenhaErrada_eRejeitado</code>	Login com senha errada devolve 401 .
<code>registroComEmailDuplicado_eRejeitado</code>	Cadastrar e-mail repetido devolve 409 .

Tipos de teste — a diferença (para explicar na banca)

	Unitário	Integração
Escopo	Uma classe isolada (<code>JwtService</code>).	Várias camadas juntas (Controller → Service → Repository → Banco).
Sobe o Spring?	Não.	Sim (<code>@SpringBootTest</code>).
Usa banco?	Não.	Sim (H2 em memória).
Velocidade	Muito rápido.	Mais lento (sobe o contexto).
Prova	A lógica de uma peça.	Que as peças funcionam juntas .

Nota técnica: por que 401 e não 403

Durante a escrita dos testes, identificamos que o Spring Security, por padrão, devolvia **403 (Forbidden)** para requisições **sem token** em rotas protegidas. O correto, segundo o HTTP, é:

- **401 Unauthorized** → não autenticado (sem token / token inválido);
- **403 Forbidden** → autenticado, mas sem permissão.

Ajustamos o `SecurityConfig` para devolver **401** quando não há autenticação (via `authenticationEntryPoint`). Isso, além de semanticamente correto, **alinha o backend com o frontend**, cujo interceptor do Axios trata o 401 para redirecionar ao login quando o token expira. O teste `fluxoCompletoDeAutenticacao` trava esse comportamento (se alguém quebrar, o teste falha).

Lançamento Automático de Renda Mensal

Visão Geral

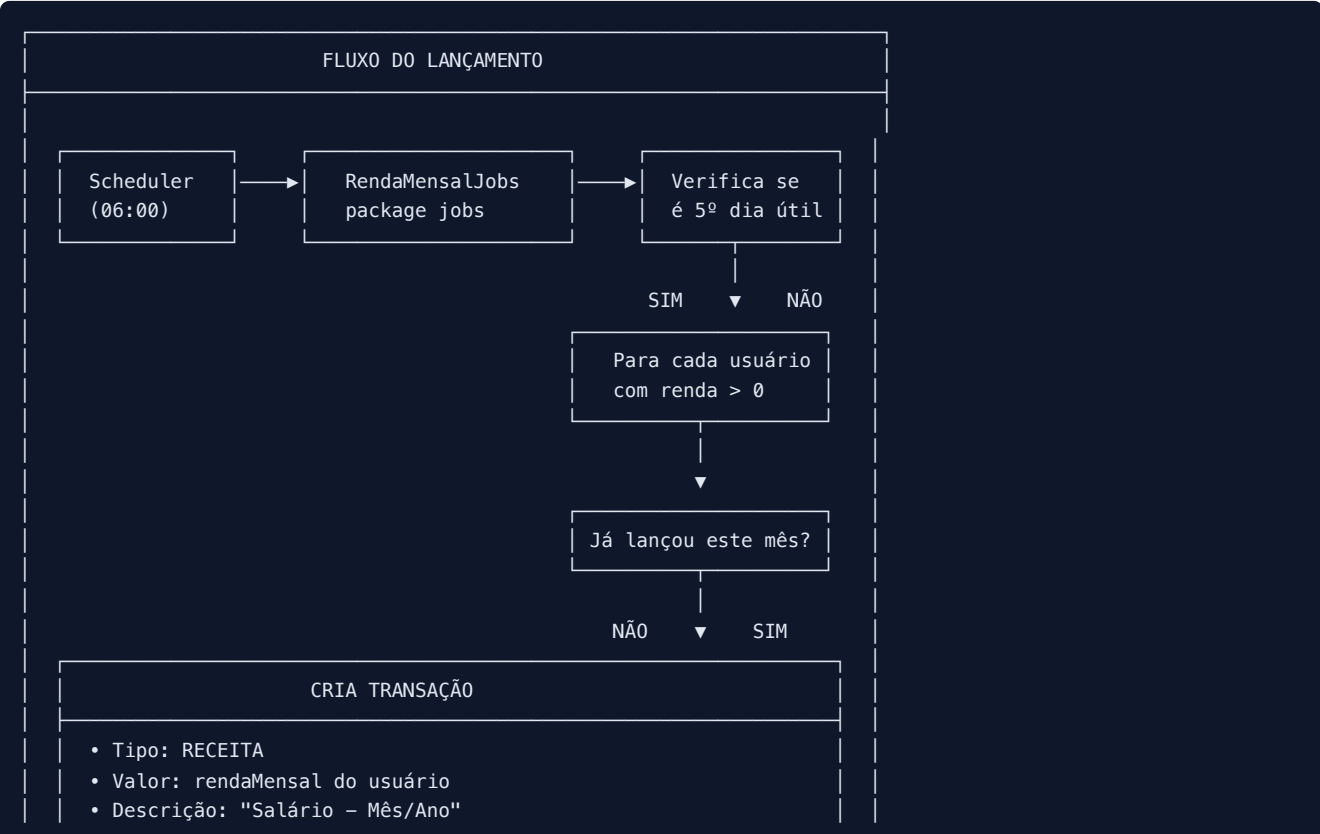
Esta funcionalidade implementa o **lançamento automático da renda mensal** cadastrada no perfil do usuário como uma transação de **RECEITA** no **5º dia útil** de cada mês, simulando o dia típico de recebimento de salário.

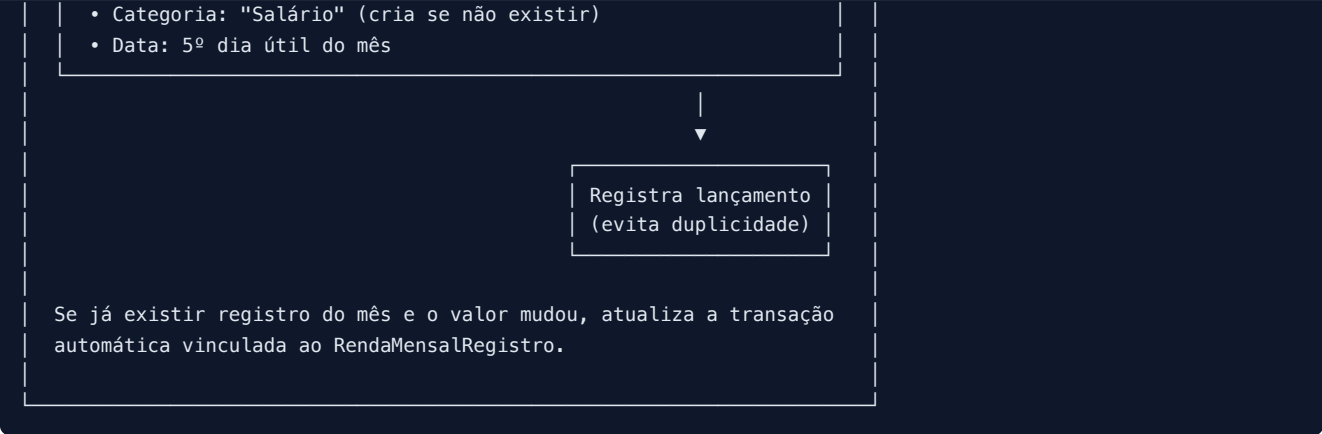
Além da execução agendada, a mesma rotina também pode ser disparada quando o usuário cria a conta com renda mensal, salva a renda no perfil ou clica no botão **Lançar renda do mês** na tela **Meu perfil**. Isso permite atualizar o Dashboard imediatamente sem esperar o próximo agendamento.

Regra de Negócio

Aspecto	Descrição
Quando executa	Todo 5º dia útil do mês, às 06:00
Dias úteis	Segunda a sexta-feira (sábado e domingo não contam)
Condição	Usuário deve ter <code>rendaMensal > 0</code> configurada no perfil
Categoria	Criada automaticamente como "Salário" (tipo RECEITA)
Controle	Registra cada lançamento para evitar duplicidade
Atualização	Se a renda mudar no perfil, atualiza a transação automática do mês
Execução manual	Botão no perfil chama a rotina apenas para o usuário logado

Arquitetura da Solução





Arquivos Criados/Modificados

1. Entidade: RendaMensualRegistro.java

Caminho: backend/src/main/java/com/financas/entity/RendaMensualRegistro.java

Propósito: Controlar se a renda mensal já foi lançada em um determinado mês/ano.

Atributos:

Atributo	Tipo	Descrição
id	UUID	Identificador único (herdado de BaseEntity)
usuario	Usuario	Usuário dono do registro
ano	Integer	Ano do lançamento (ex: 2026)
mes	Integer	Mês do lançamento (1-12)
valorLancado	BigDecimal	Valor que foi lançado
dataLancamento	LocalDate	Data em que o lançamento foi feito
transacao	Transacao	Referência à transação criada

Constraint: uk_renda_usuario_ano_mes - Garante unicidade por usuário/ano/mês

2. Repository: RendaMensualRegistroRepository.java

Caminho: backend/src/main/java/com/financas/repository/RendaMensualRegistroRepository.java

Métodos:

Método	Retorno	Descrição
existsByUsuarioIdAndAnoAndMes(UUID, Integer, Integer)	boolean	Verifica se já existe lançamento no mês
findByUsuarioIdAndAnoAndMes(UUID, Integer, Integer)	Optional	Busca registro específico
findByUsuarioIdOrderByAnoDescMesDesc(UUID)	List	Lista histórico do usuário

3. Job: RendaMensualJobs.java

Caminho: backend/src/main/java/com/financas/jobs/RendaMensualJobs.java

Métodos principais:

Método	Descrição
<code>processarRendaMensual()</code>	Rotina agendada (cron: <code>0 0 6 * * ?</code>)
<code>executarParaUsuario(Usuario, LocalDate)</code>	Cria ou atualiza o lançamento somente para um usuário
<code>ehQuintoDiaUtil(LocalDate)</code>	Verifica se a data é o 5º dia útil
<code>calcularQuintoDiaUtil(int, int)</code>	Calcula o 5º dia útil de um mês
<code>executarManualmente(LocalDate)</code>	Execução manual geral para todos os usuários com renda
<code>buscarOuCriarCategoriaSalario(Usuario)</code>	Cria categoria "Salário" se não existir

Resultados possíveis da rotina:

Resultado	Significado
<code>CRIADO</code>	Não havia lançamento do mês e uma nova transação foi criada
<code>ATUALIZADO</code>	Já havia lançamento do mês e o valor da transação foi atualizado
<code>JA_PROCESSADO</code>	Já havia lançamento do mês com o mesmo valor
<code>SEM_RENDA</code>	Usuário não tem renda mensal maior que zero

Anotações utilizadas:

- `@Component` - Declara o job como componente Spring
- `@Scheduled(cron = "0 0 6 * * ?")` - Agenda execução diária às 06:00
- `@Transactional` - Garante atomicidade das operações
- `@Slf4j` - Habilita logs (Lombok)

4. Controller: `RendaMensualController.java`

Caminho: `backend/src/main/java/com/financas/controller/RendaMensualController.java`

Endpoints:

Método	Endpoint	Descrição
GET	<code>/api/renda-mensual/status</code>	Status do lançamento automático
POST	<code>/api/renda-mensual/executar</code>	Cria ou atualiza lançamento para o usuário logado
GET	<code>/api/renda-mensual/historico</code>	Lista histórico de lançamentos

5. DTO: `RendaMensualStatusResponse.java`

Caminho: `backend/src/main/java/com/financas/dto/RendaMensualStatusResponse.java`

Atributos:

Atributo	Tipo	Descrição
<code>rendaMensualConfigurada</code>	BigDecimal	Valor configurado no perfil
<code>lancadoEsteMes</code>	boolean	Se já foi lançado este mês
<code>proximoLancamento</code>	LocalDate	Data do próximo 5º dia útil
<code>totalLancamentos</code>	int	Total de lançamentos realizados

6. Modificação: `FinancasApplication.java`

Adicionado: `@EnableScheduling`

```
@SpringBootApplication
@EnableScheduling // <-- ADICIONADO
public class FinancasApplication {
```

Propósito: Habilita o agendamento de tarefas no Spring Boot.

7. Integração com Cadastro e Perfil

Arquivos relacionados:

Arquivo	Responsabilidade
<code>backend/src/main/java/com/financas/service/AuthService.java</code>	Ao criar a conta, chama <code>RendaMensalJobs.executarParaUsuario(...)</code> se houver renda
<code>backend/src/main/java/com/financas/service/UsuarioService.java</code>	Ao salvar o perfil, chama <code>RendaMensalJobs.executarParaUsuario(...)</code>
<code>frontend/src/pages/Perfil.tsx</code>	Exibe o botão Lançar renda do mês
<code>frontend/src/services/servicoRendaMensal.ts</code>	Chama <code>POST /api/renda-mensal/executar</code>

O lançamento continua idempotente: se a renda já foi lançada no mês, a rotina não cria outra transação. Quando o valor da renda mensal muda no perfil, a rotina atualiza o `valor` do `RendaMensalRegistro` e da `Transacao` automática vinculada.

Cálculo do 5º Dia Útil

Algoritmo

```
public LocalDate calcularQuintoDiaUtil(int ano, int mes) {
    LocalDate data = LocalDate.of(ano, mes, 1);
    int diasUteis = 0;

    while (diasUteis < 5) {
        if (ehDiaUtil(data)) {
            diasUteis++;
        }
        if (diasUteis < 5) {
            data = data.plusDays(1);
        }
    }

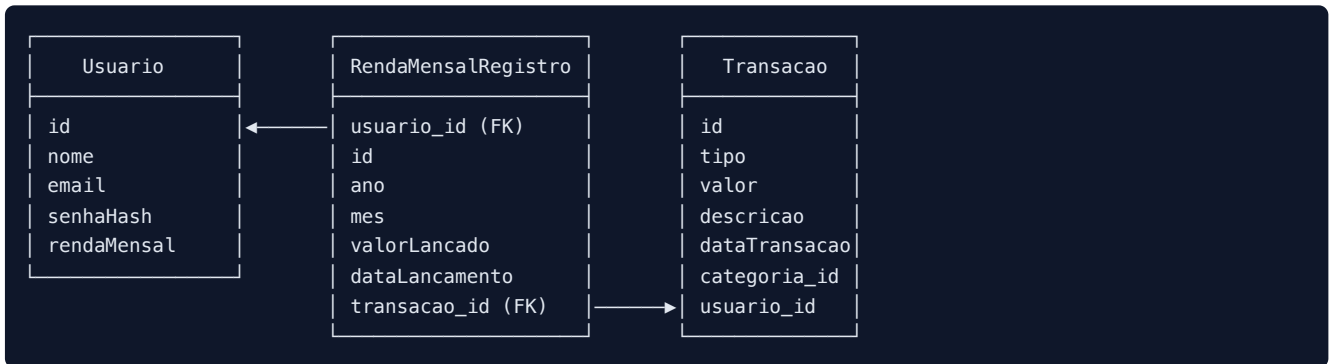
    return data;
}

private boolean ehDiaUtil(LocalDate data) {
    DayOfWeek dia = data.getDayOfWeek();
    return dia != DayOfWeek.SATURDAY && dia != DayOfWeek.SUNDAY;
}
```

Exemplos

Mês/Ano	5º Dia Útil	Explicação
Janeiro/2026	07/01/2026	Dia 1 (qui), 2 (sex), 5 (seg), 6 (ter), 7 (qua)
Fevereiro/2026	06/02/2026	Dia 2 (seg), 3 (ter), 4 (qua), 5 (qui), 6 (sex)
Março/2026	05/03/2026	Dia 2 (seg), 3 (ter), 4 (qua), 5 (qui), 6 (sex) - ajustado

Diagrama de Entidades



Logs Gerados

A rotina gera logs detalhados para monitoramento:

```

[RendaMensal] Iniciando verificacao - Data: 2026-06-05
[RendaMensal] Hoje e o 5o dia util! Processando lancamentos...
[RendaMensal] Usuarios com renda configurada: 3
[RendaMensal] Lancamento realizado: Usuario=joao@email.com, Valor=5000.00, Mes=6/2026
[RendaMensal] Lancamento atualizado: Usuario=joao@email.com, Valor=5500.00, Mes=6/2026
[RendaMensal] Renda ja lancada para usuario maria@email.com em 6/2026
[RendaMensal] Processamento concluido. Lancamentos/atualizacoes realizados: 1
[RendaMensal] Execucao solicitada para usuario joao@email.com na data: 2026-06-05
  
```

Como Testar

1. Via API (Postman/Swagger)

```

# Ver status
GET http://localhost:8080/api/renda-mensal/status

# Executar manualmente para o usuário logado
POST http://localhost:8080/api/renda-mensal/executar

# Executar para uma data específica para o usuário logado
POST http://localhost:8080/api/renda-mensal/executar?data=2026-07-07

# Ver histórico
GET http://localhost:8080/api/renda-mensal/historico
  
```

2. Via Frontend

Na tela **Meu perfil**, informe uma renda mensal maior que zero e clique em **Lançar renda do mês**.

Se o salário já estiver lançado no mês e você alterar a renda no perfil, a transação automática em **Transações** também terá o valor atualizado.

3. Verificar no Dashboard

Após executar, a receita aparecerá:

- No Dashboard como "Receita do Mês"
- Na lista de Transações com descrição "Salário - Mês/Ano"

Integração com o Frontend

O frontend exibe a renda mensal no **Painel (Dashboard)** como parte das receitas do mês. Após o lançamento automático, salvamento do perfil, cadastro com renda ou clique no botão do perfil:

1. A transação é criada ou atualizada no banco de dados
2. O Dashboard calcula receitas/despesas do período
3. O valor da renda aparece como receita

Arquivos relacionados: `frontend/src/pages/Painel.tsx`, `frontend/src/pages/Perfil.tsx`, `frontend/src/services/servicoRendaMensal.ts`

Considerações Técnicas

Fuso Horário

- O scheduler usa o fuso horário do servidor
- Recomendado configurar `TZ=America/Sao_Paulo` em produção

Feriados

- Esta versão **não considera feriados nacionais**
- Apenas sábados e domingos são ignorados
- Possível melhoria futura: integrar API de feriados

Performance

- A rotina executa uma vez por dia (06:00)
- Processa apenas usuários com `rendaMensal > 0`
- Usa índice único para evitar duplicidade
- Atualiza apenas a transação vinculada ao `RendaMensalRegistro`; receitas manuais não são alteradas

Resumo dos Arquivos

Arquivo	Tipo	Ação
<code>RendaMensalRegistro.java</code>	Entity	CRIADO
<code>RendaMensalRegistroRepository.java</code>	Repository	CRIADO
<code>RendaMensalJobs.java</code>	Job	CRIADO/MOVIDO
<code>RendaMensalController.java</code>	Controller	CRIADO

Arquivo	Tipo	Ação
RendaMensalStatusResponse.java	DTO	CRIADO
FinancasApplication.java	Application	MODIFICADO (adicionado @EnableScheduling)
AuthService.java	Service	MODIFICADO (dispara job no cadastro)
UsuarioService.java	Service	MODIFICADO (dispara job ao salvar perfil)
Perfil.tsx	Frontend	MODIFICADO (botão para lançar renda do mês)
servicoRendaMensal.ts	Frontend Service	CRIADO

Autor

Sistema Finanças Simples - TCC 2026